

```
pidstat -d -h -r -u -p 1234 5
```

...where 1234 is the PID of the process to be monitored and 5 is the delay in seconds.

—Prasanna Mohanasundaram,
prasanna.mohanasundaram@gmail.com

Finding the number of threads a process has

By using the `ps` command and reading the `/proc` file entries, we can find the number of threads in a process.

Here, the `nlwp` option will report the number of light weight processes (i.e., threads).

```
(bash)$ ps -o nlwp <PID Of Process>
```

```
(bash)$ ps -o nlwp 3415
```

```
NLWP
```

```
34
```

We can also obtain the same information by reading `/proc` file entries:

```
(bash)$ cat /proc/<PID>/status | grep Threads
```

```
(bash)$ cat /proc/3415/status | grep Threads
```

```
Threads: 34
```

—Narendra Kangralkar,
narendrakangralkar@gmail.com

Find out the number of times a user has executed a command

In Linux, the command known as `hash` will display the number of hits or number of times that a particular shell command has been executed.

```
test@linux-erv3:~/Desktop> hash
hits command
1 /bin/ps
2 /usr/bin/pidstat
1 /usr/bin/man
2 /usr/bin/top
test@linux-erv3:~/Desktop>
```

In the above output of `hash`, we can see that the commands `pidstat` and `top` have been used twice and the rest only once.

If `hash` is the first command in the terminal, it will return the following as output:

```
test@linux-erv3:~/Desktop> hash
```

```
hash: hash table empty
```

```
test@linux-erv3:~/Desktop>
```

—Bharanidharan Ambalavanan,
abharanidharan@srec.ac.in

Searching for a specific string entirely within a particular location

If you're working on a big project with multiple files organised in various hierarchies and would need to know where a specific string/word is located, the following Grep command option would prove to be priceless.

```
grep -HRnF 'foo test' <dir_loc> --include "*.c"
```

The above command will search for `'foo test'` (without the quotes) under `<dir_loc>` recursively for all the files ending with `.c` extension and will print the output with the line number.

Remember that `'foo test'` is case sensitive, so a search for `'Foo Test'` would return different results.

Here is a sample output:

```
<dir_loc>/tip/src/WWW/cgi-bin/admin/file1.c:880: line
beginning of foo test in sample;
```

```
<dir_loc>/tip/src/WWW/cgi-bin/admin/file1.c:1034: foo test
line ending;
```

```
<dir_loc>/tip/src/WWW/cgi-bin/user/file2.c:166: partial foo
test;
```

```
<dir_loc>/file3.c:176: somewhere foo testing of sample;
```

You can always replace `<dir_loc>` with `(.dot)` to indicate a recursive search from the current directory and also leave out the `--include` parameter to search for all files; or provide it as `--include "*.c"`

```
grep -HRnF 'foo test' . --include "*.c"
```

—Runcy Oommen,
runcy.oommen@gmail.com

END ->

Share Your Linux Recipes!

The joy of using Linux is in finding ways to get around problems—take them head on, defeat them! We invite you to share your tips and tricks with us for publication in *OSFY* so that they can reach a wider audience. Your tips could be related to administration, programming, troubleshooting or general tweaking. Submit them at www.opensourceforu.com. The sender of each published tip will get a T-shirt.